

Appendix

The working: generated analysis output, every query, and the verification run.

Contents

- 01 What is in this appendix

- 02 Verification run — 91 assertions, all passing

- 03 Part C Task 1 — the full release analysis

- 04 Decisions settled before the tiers were chosen

- 05 Supplementary tests — CSAT, refund gate, data quality

- 06 Cohort baselines and the stratified control test

- 07 Part A — k-modes segment clustering

- 08 SQL · 00_explore_schema

- 09 SQL · 01_engagement_tiers

- 10 SQL · 02_core_metrics

- 11 SQL · 03_segment_breakdown

- 12 SQL · 10_explore_events

- 13 SQL · 11_challenge_funnel

- 14 SQL · 12_challenge338_user_table

- 15 SQL · 13_comparison_groups

- 16 Code · Verification — re-derives every published figure

- 17 Code · Part D — the LTV and A/B model

- 18 Code · Part A — the segment clustering

What is in this appendix

Everything here is **generated output**, not hand-written prose. If any figure in the submission, the dossier or the deck is ever questioned, the answer is in one of these files — and all of it can be regenerated from the source CSVs in one command.

```
cd part-c-release-verdict/analysis
python3 05_qa.py          # re-derives all 91 published figures and fails loudly on any drift
```

Contents

FILE	WHAT'S IN IT	ANSWERS
05_qa-output.txt	91 assertions , each computed from the raw CSV and checked against the published figure	<i>"Is this number real?" — start here</i>
02_main-output.txt	The full Part C Task-1 analysis: exposure ladder, tier calibration, the headline comparison with confidence intervals, all four causal tests, threshold-robustness sweep, and every segment cut	<i>"Show your calculations"</i>
03_supplement-output.txt	Target-metric definitions side by side, the CSAT inversion, the refund-gate test, the popup-decliner group, data-quality flags, and the economic sizing of the feature	<i>"What else did you check?"</i>
04_loose_ends-output.txt	Cohort baselines, the confound check on exposure, the stratified takers-vs-control comparison , challenge-start dates, and the plan-mix validation against Part D	<i>"Is the effect just activity level?"</i>
01_diagnostics-output.txt	The questions settled <i>before</i> the engagement tiers were chosen: challenge-start lag, the 8-lesson structure, dose-response by both measures, censoring, and coverage of course 338	<i>"How did you pick the cut?"</i>
sql/	All 8 BigQuery queries — see the table below for which produced data	<i>"How did you pull it?"</i>

The queries, and which ones are load-bearing

00 probes the real schema; **01** – **03** were written **before** it was known, against assumed column names, and were superseded rather than run — they are kept because they are the actual working, and they still carry their **△ADJUST** markers. Five queries produced every CSV the analysis reads.

QUERY	RAN	PRODUCED
00_explore_schema.sql	✓	95 rows — the real column list for both tables
01_engagement_tiers.sql	—	superseded draft (assumed schema)
02_core_metrics.sql	—	superseded draft (assumed schema)
03_segment_breakdown.sql	—	superseded draft (assumed schema)
10_explore_events.sql	✓	the 67-event catalogue, event×course counts, unsubscribe-reason counts, row samples

QUERY	RAN	PRODUCED
11_challenge_funnel.sql	✓	1 row — the whole-cohort funnel counts
12_challenge338_user_table.sql	✓	1,111 rows — one per challenge-338 starter
13_comparison_groups.sql	✓	9,956 rows — one per paying subscriber. The analysis dataset
14_unsub_timing.sql	not run	would make unsub % at 12h/24h reportable

The three numbers most likely to be challenged

CLAIM	WHERE TO FIND IT	TEST
The Challenge moved D1 by +0.5 pts, p = 0.82	02_main-output.txt §4 Test A	Two-proportion z-test, takers (n=1,111) vs exposure-matched control (n=994)
The effect is not an activity artefact	04_loose_ends-output.txt	Stratified by product activity — flat in 4 of 5 strata
27% of finishers completed it in one sitting	02_main-output.txt §5	13 of 48 finishers have <code>active_challenge_days = 1</code>

Method notes

- **Confidence intervals** are Wilson score intervals, not normal-approximation — several cells are small and several proportions sit near 0 or 1, where the normal approximation returns impossible bounds.
- **Group comparisons** are two-proportion z-tests with a pooled-variance standard error, reported as difference, z and p rather than as a bare "significant".
- **Censoring** is handled with explicit restricted denominators, stated on every table. The observation window is 14 days (2026-06-12 → 2026-06-25), so D7 is only observable for 72.5% of the cohort and **every unsubscribe rate is a floor**.
- **No regression, no model**. With no hold-out group in the data, the honest tool is a natural control — people who reached the same surface and declined it. Simpler than a model, and it does not hide its assumptions inside coefficients.
- The analysis scripts are **pure Python standard library** (no pandas or numpy on the machine they were written on), so they run anywhere with Python 3 and the two CSVs.

What this appendix cannot tell you

Whether the Challenge was *causal*. There is no hold-out group and no pre/post period anywhere in the warehouse — the release shipped to 100%. Every causal statement in the submission is an observational correction that worked only because a large "looked at it and left" group happened to exist as a natural control. Fixing that is the first recommendation in Part C, not the last.

Verification run — 91 assertions, all passing

```
== cohort shape ==
  ok subscribers in cohort          9956.0000 (doc: 9956)
  ok challenge-338 starters         1111.0000 (doc: 1111)
  ok non-takers                    8845.0000 (doc: 8845)
  ok all rows have a subscription  9956.0000 (doc: 9956)

== exposure ladder (Part C §1) ==
  ok saw popup                     3140.0000 (doc: 3140)
  ok saw popup %                   0.3154 (doc: 0.315)
  ok clicked popup                 2970.0000 (doc: 2970)
  ok viewed a challenge            2109.0000 (doc: 2109)
  ok started 338 %                 0.1116 (doc: 0.112)
  ok never reached a surface       6716.0000 (doc: 6716)
  ok never reached a surface %     0.6746 (doc: 0.675)

== inside the challenge (Part C §2) ==
  ok completed >= 1 lessons        612.0000 (doc: 612)
  ok completed >= 2 lessons        308.0000 (doc: 308)
  ok completed >= 3 lessons        158.0000 (doc: 158)
  ok completed >= 8 lessons        47.0000 (doc: 47)
  ok zero lessons %               0.4491 (doc: 0.449)
  ok lesson1->2 survival           0.5033 (doc: 0.503)
  ok completed the course          48.0000 (doc: 48)
  ok certificate                   23.0000 (doc: 23)

== tiers + headline table (Part C §4) ==
  ok HIGH n                       158.0000 (doc: 158)
  ok LOW n                        953.0000 (doc: 953)
  ok HIGH D1                      0.6962 (doc: 0.696)
  ok LOW D1                       0.5845 (doc: 0.584)
  ok DIDNT-TAKE D1                0.2174 (doc: 0.217)
  ok HIGH D3                      0.5696 (doc: 0.57)
  ok LOW D3                       0.3442 (doc: 0.344)
  ok DIDNT-TAKE D3                0.1001 (doc: 0.1)
  ok HIGH D7                      0.3910 (doc: 0.391)
  ok HIGH D7 denominator          133.0000 (doc: 133)
  ok LOW D7                       0.1805 (doc: 0.18)
  ok LOW D7 denominator           737.0000 (doc: 737)
  ok DIDNT-TAKE D7                0.0689 (doc: 0.069)
  ok DIDNT-TAKE D7 denominator    6345.0000 (doc: 6345)
  ok HIGH unsub                   0.1139 (doc: 0.114)
  ok LOW unsub                    0.1144 (doc: 0.114)
  ok DIDNT-TAKE unsub             0.1431 (doc: 0.143)

== the decisive test (Part C §5 Test A) ==
  ok takers D1                    0.6004 (doc: 0.6)
  ok control n                    994.0000 (doc: 994)
  ok control D1 (the 59.6%)        0.5956 (doc: 0.596)
  ok D1 effect (pts)               0.0048 (doc: 0.005)
  ok D1 effect p-value             0.8231 (doc: 0.823)
  ok unsub effect (pts)            -0.0225 (doc: -0.023)
  ok unsub effect p-value          0.1189 (doc: 0.119)
  ok joined-never-started D1       0.6043 (doc: 0.604)
  ok viewed-never-joined D1        0.5929 (doc: 0.593)

== immortal time + dose-response (Test B / C) ==
  ok start on day 0 %              0.2394 (doc: 0.239)
  ok day-0 dose effect (pts, negative) -0.0920 (doc: -0.092)
  ok day-0 dose p-value            0.0475 (doc: 0.048)

== the mechanic (Test D) ==
  ok finished in 1 day             13.0000 (doc: 13)
  ok finished in 1 day %           0.2708 (doc: 0.271)
  ok finished in <=2 days %        0.4583 (doc: 0.458)
```

ok	returned for a 2nd challenge day %	0.3676	(doc: 0.368)
ok	mean lessons per active challenge day	1.4608	(doc: 1.46)
== the metric that would have fooled the team (§6) ==			
ok	takers D1, challenge-anchored	0.3978	(doc: 0.398)
ok	cohort D1, first-app-day	0.2602	(doc: 0.26)
== CSAT inversion (§8) ==			
ok	CSAT never took	3.7373	(doc: 3.74)
ok	CSAT LOW	3.5338	(doc: 3.53)
ok	CSAT HIGH	3.3332	(doc: 3.33)
ok	CSAT finishers	3.1040	(doc: 3.1)
ok	CSAT of challenge-338 content	3.2543	(doc: 3.25)
ok	same users, all content	3.4504	(doc: 3.45)
== product-wide baselines (the thesis number) ==			
ok	never completed a lesson anywhere %	0.5021	(doc: 0.502)
ok	cohort unsubscribe %	0.1399	(doc: 0.14)
== demographics (Part A + the PM's claim) ==			
ok	male %	0.5974	(doc: 0.597)
ok	female %	0.3841	(doc: 0.384)
ok	aged 45+ (clean buckets) %	0.5776	(doc: 0.578)
ok	aged 45+ incl. legacy bucket %	0.6012	(doc: 0.601)
ok	men 55+ count	2120.0000	(doc: 2120)
ok	men 55+ %	0.2129	(doc: 0.213)
ok	men 35-54 %	0.2495	(doc: 0.249)
ok	unsub, age 18-24	0.3111	(doc: 0.311)
ok	unsub, age 25-34	0.1970	(doc: 0.197)
ok	unsub, age 35-44	0.1447	(doc: 0.145)
ok	unsub, age 45-54	0.0953	(doc: 0.095)
ok	unsub, age 55+	0.0969	(doc: 0.097)
ok	unsub, goal 'Work faster'	0.1095	(doc: 0.109)
ok	unsub, goal 'Feel more confident with AI'	0.1074	(doc: 0.107)
ok	unsub, goal 'Get a quick side hustle'	0.3723	(doc: 0.372)
== plans (Part D validation) ==			
ok	1Week share	0.1018	(doc: 0.102)
ok	4Week share incl. special	0.6459	(doc: 0.646)
ok	12Week share	0.2506	(doc: 0.251)
ok	1Week unsub	0.3462	(doc: 0.346)
ok	4Week unsub	0.1219	(doc: 0.122)
ok	12Week unsub	0.1058	(doc: 0.106)
ok	observed-mix blended net LTV	126.8996	(doc: 126.9)
== unsubscribe reasons (§8) ==			
ok	total unsubscribed	1393.0000	(doc: 1393)
ok	payment/chargeback/dispute unsubs	157.0000	(doc: 157)
ok	... as % of unsubs	0.1127	(doc: 0.113)
== event catalogue (the 'not observable here' claim) ==			
ok	event catalogue rows (file integrity)	67.0000	(doc: 67)
ok	subscription_renewed events	17.0000	(doc: 17)
ok	... per subscriber	0.0017	(doc: 0.0017)

=====

All 91 checks passed – every documented number matches the data.

Part C Task 1 — the full release analysis

§1 · THE EXPOSURE LADDER — of every paying subscriber, who ever reaches the Challenge?

step	n	% of cohort	step conv.
Paying subscribers in the cohort	9956	100.0%	—
... saw the Challenge popup	3140	31.5%	31.5%
... clicked the popup	2970	29.8%	94.6%
... viewed a Challenge page	2109	21.2%	71.0%
... joined Challenge 338	1341	13.5%	63.6%
... STARTED Challenge 338	1111	11.2%	82.8%
... completed ≥1 challenge lesson	613	6.2%	55.2%
... completed ≥3 challenge lessons	158	1.6%	25.8%
... completed the whole challenge	48	0.5%	30.4%
... took the certificate	23	0.2%	47.9%

Where the audience is lost (share of the whole paying cohort):

never reached any challenge surface : 6716 (67.5%)
reached a surface, never started 338: 2129 (21.4%)
started 338 : 1111 (11.2%)

§2 · ENGAGEMENT TIERS — definition and calibration

The Challenge ships 8 lessons (confirmed: every user flagged 'course_completed' has exactly 8 completions). So 'finished' = 8, and the day-3 content sits at lesson 3.

Tier rule (stated up front, applied everywhere below):

HIGH started 338 AND completed ≥ 3 of the 8 lessons (got past the Day-2 cliff)
LOW started 338 AND completed ≤ 2 lessons
DIDN'T TAKE never started 338

The cut is at 3 because that is where the D7 curve steps (18% → 39%) and it matches the product's own claim ('one day = one skill' → 3 lessons = a 3-day habit). §4 tests it for robustness at every other threshold.

Challenge-lesson completion histogram (n=1111 starters):

0 lessons	499	44.9%	cum 44.9%	
1 lessons	304	27.4%	cum 72.3%	
2 lessons	150	13.5%	cum 85.8%	
3 lessons	53	4.8%	cum 90.5%	
4 lessons	25	2.3%	cum 92.8%	
5 lessons	18	1.6%	cum 94.4%	
6 lessons	10	0.9%	cum 95.3%	
7 lessons	5	0.5%	cum 95.8%	
8 lessons	47	4.2%	cum 100.0%	

Lesson-by-lesson survival INSIDE the challenge (of the 1,111 who pressed start):

reached lesson	n	% of starters	survived prev. step
completed ≥ 1	612	55.1%	55.1%
completed ≥ 2	308	27.7%	50.3%
completed ≥ 3	158	14.2%	51.3%
completed ≥ 4	105	9.5%	66.5%
completed ≥ 5	80	7.2%	76.2%
completed ≥ 6	62	5.6%	77.5%
completed ≥ 7	52	4.7%	83.9%
completed ≥ 8	47	4.2%	90.4%

Tier sizes:

HIGH (3+ lessons) 158 (1.6% of subscribers)
LOW (0-2 lessons) 953 (9.6% of subscribers)
DIDN'T TAKE IT 8845 (88.8% of subscribers)

§3 · THE HEADLINE COMPARISON (retention anchored on each user's FIRST APP DAY for all three groups, so the groups are on the same clock – see §4 for why this matters)

metric	HIGH (3+ lessons)	LOW (0–2 lessons)	DIDN'T TAKE IT
D1 retention ★TARGET	69.6% [62.1%–76.3%]	58.4% [55.3%–61.5%]	21.7% [20.9%–22.6%]
D3 retention	57.0% [49.2%–64.4%]	34.4% [31.5%–37.5%]	10.0% [9.4%–10.6%]
D7 retention	39.1% [31.2%–47.6%]	18.0% [15.4%–21.0%]	6.9% [6.3%–7.5%]
Unsubscribe rate (D7 denominator)	11.4% [7.3%–17.3%] 133	11.4% [9.6%–13.6%] 737	14.3% [13.6%–15.1%] 6345
Avg lessons completed (all)	19.28	6.38	1.81
Avg active days in product	5.37	3.46	1.76
Avg CSAT (1–5) (CSAT raters)	3.33 158	3.53 845	3.74 3740

Significance vs 'didn't take it':

HIGH (3+ lessons)	D1	diff	+47.9%	z=+14.27	p=0.00e+00	***
HIGH (3+ lessons)	D3	diff	+47.0%	z=+18.83	p=0.00e+00	***
HIGH (3+ lessons)	D7	diff	+32.2%	z=+13.92	p=0.00e+00	***
HIGH (3+ lessons)	unsub	diff	-2.9%	z= -1.04	p=2.98e-01	ns
LOW (0–2 lessons)	D1	diff	+36.7%	z=+24.76	p=0.00e+00	***
LOW (0–2 lessons)	D3	diff	+24.4%	z=+21.74	p=0.00e+00	***
LOW (0–2 lessons)	D7	diff	+11.2%	z=+10.54	p=0.00e+00	***
LOW (0–2 lessons)	unsub	diff	-2.9%	z= -2.43	p=1.52e-02	*

§4 · IS THE GAP CAUSAL? – three tests

TEST A – EXPOSURE-MATCHED CONTROL.

'Didn't take it' is not one group: most of them were never offered the Challenge.
The fair control is people who DID reach the Challenge and chose not to engage.

exposure group	n	D1	D3	unsub
started 338 (takers)	1111	60.0% [57.1%–62.9%]	37.6% [34.8%–40.5%]	11.4% [9.7%–13.4%]
joined 338, never started	230	60.4% [54.0%–66.5%]	37.8% [31.8%–44.2%]	13.9% [10.0%–19.0%]
viewed 338, never joined	764	59.3% [55.8%–62.7%]	36.6% [33.3%–40.1%]	13.6% [11.4%–16.2%]
saw popup, never opened 338	1132	44.9% [42.0%–47.8%]	24.0% [21.6%–26.6%]	25.5% [23.1%–28.1%]
never reached a surface	6716	12.2% [11.4%–13.0%]	3.7% [3.2%–4.1%]	12.5% [11.7%–13.3%]

takers vs exposed-but-didn't-start: D1 diff +0.5% z=+0.22 p=0.823 ns
takers vs exposed-but-didn't-start: unsub diff -2.3% z=-1.56 p=0.119 ns

TEST B – IMMORTAL-TIME BIAS.

Only 24% of takers start on their first app day; the median taker starts on day +1.
To start on day +k you must already have survived to day k – so takers are pre-selected survivors. The clean read is the 'day-0 starters' who had no survival head start.

taker start-day lag: day0 266 (24%), day+1 342 (31%), day+2 196 (18%), day+3 or later 307 (28%)

group	n	D1	D3
takers who started on day 0	266	39.1% [33.4%–45.1%]	22.6% [17.9%–27.9%]
all takers (any start day)	1111	60.0% [57.1%–62.9%]	37.6% [34.8%–40.5%]
never took it	8845	21.7% [20.9%–22.6%]	10.0% [9.4%–10.6%]

day-0 takers vs never-took: D1 diff +17.4% z=+6.71 p=2.00e-11 ***

TEST C – A NON-TAUTOLOGICAL DOSE-RESPONSE.

'More lessons -> better retention' is partly circular: doing lesson 3 on day 3 IS retention.
The clean version: among users whose challenge activity happened on ONE day only, does a bigger DAY-0 dose predict coming back tomorrow? Nothing circular here.

n = 816 takers with all challenge activity on a single day

day-0 lessons	n	D1 next day	unsub
---------------	---	-------------	-------

0	429	55.7%	[51.0%-60.3%]	10.5%	[7.9%-13.7%]
1	248	54.4%	[48.2%-60.5%]	12.5%	[8.9%-17.2%]
2	99	50.5%	[40.8%-60.1%]	11.1%	[6.3%-18.8%]
3	21	33.3%	[17.2%-54.6%]	23.8%	[10.6%-45.1%]
4	5	0.0%	[0.0%-43.4%]	20.0%	[3.6%-62.4%]
8	13	53.8%	[29.1%-76.8%]	38.5%	[17.7%-64.5%]

>=2 vs <=1 day-0 lessons: D1 diff -9.2% z=-1.98 p=0.048 *

THRESHOLD ROBUSTNESS – the 'high' cut at every possible value:

high = >= k lessons	n high	D1 high	D1 low	D1 gap	unsub high	unsub low
1	612	61.4%	58.3%	+3.1%	12.7%	9.8%
2	308	63.3%	58.8%	+4.5%	13.0%	10.8%
3	158	69.6%	58.4%	+11.2%	11.4%	11.4%
4	105	74.3%	58.5%	+15.7%	10.5%	11.5%
5	80	78.8%	58.6%	+20.2%	10.0%	11.5%
6	62	80.6%	58.8%	+21.8%	11.3%	11.4%
7	52	76.9%	59.2%	+17.7%	11.5%	11.4%
8	47	76.6%	59.3%	+17.3%	12.8%	11.4%

§5 · THE MECHANIC CHECK – is this actually a DAILY challenge?

The hypothesis was 'one day = one skill builds a daily habit'. That only works if the content is gated to one lesson per day. Test: how many lessons do people do per active day?

Users who completed the whole 8-lesson challenge: 48 (4.3% of starters, 0.48% of subscribers)

... and how many DAYS they took to do it:

1 day(s):	13	(27.1%)	cum 27.1%	██████████
2 day(s):	9	(18.8%)	cum 45.8%	██████████
3 day(s):	6	(12.5%)	cum 58.3%	██████████
4 day(s):	4	(8.3%)	cum 66.7%	██████████
5 day(s):	5	(10.4%)	cum 77.1%	██████████
6 day(s):	2	(4.2%)	cum 81.2%	██████████
7 day(s):	3	(6.2%)	cum 87.5%	██████████
8 day(s):	3	(6.2%)	cum 93.8%	██████████
9 day(s):	2	(4.2%)	cum 97.9%	██████████
10 day(s):	1	(2.1%)	cum 100.0%	██████████

→ 13/48 (27%) finished the '7-day' challenge in ONE day.

→ 22/48 (46%) finished it in two days or fewer.

Lessons completed per active challenge day (all takers who did >=1 lesson):

~0 lessons/day:	63	(10.3%)
~1 lessons/day:	357	(58.3%)
~2 lessons/day:	136	(22.2%)
~3 lessons/day:	28	(4.6%)
~4 lessons/day:	14	(2.3%)
~5 lessons/day:	1	(0.2%)
~8 lessons/day:	13	(2.1%)

mean lessons per active challenge day: 1.46

Return behaviour: of takers who did >=1 lesson, how many ever came back for a 2nd challenge day?
225/612 = 36.8%

§6 · SEGMENTS

— AGE —								
segment	subs	% cohort	take rate	D1 (all)	unsub	D1 takers	D1	
non-tk								
55+	3200	32.1%	10.2%	25.4%	9.7%	60.9%		
21.4%								

45-54	2551	25.6%	10.9%	26.9%	9.5%	62.1%	
22.6%							
35-44	1887	19.0%	12.0%	27.8%	14.5%	66.1%	
22.5%							
25-34	1117	11.2%	12.4%	24.4%	19.7%	54.0%	
20.1%							
18-24	913	9.2%	10.3%	21.9%	31.1%	44.7%	
19.3%							
45+	235	2.4%	20.0%	37.0%	17.0%	59.6%	
31.4%							
(unknown)	53	0.5%	3.8%	13.2%	43.4%	100.0%	
9.8%							
— GENDER —							
segment	subs	% cohort	take rate	D1 (all)	unsub	D1 takers	D1
non-tk							

Male	5948	59.7%	11.3%	27.4%	12.5%	62.3%	
23.0%							
Female	3824	38.4%	11.0%	23.8%	16.1%	56.2%	
19.8%							
I'd rather skip this one	122	1.2%	10.7%	31.1%	6.6%	69.2%	
26.6%							
(unknown)	62	0.6%	4.8%	16.1%	38.7%	66.7%	
13.6%							
— PLAN —							
segment	subs	% cohort	take rate	D1 (all)	unsub	D1 takers	D1
non-tk							

4Week	6341	63.7%	11.2%	26.5%	12.2%	60.5%	
22.2%							
12Week	2495	25.1%	11.7%	26.4%	10.6%	61.3%	
21.8%							
1Week	1014	10.2%	9.5%	20.9%	34.6%	51.0%	
17.8%							
4Week_special	90	0.9%	12.2%	33.3%	3.3%	72.7%	
27.8%							
— WORK STATUS —							
segment	subs	% cohort	take rate	D1 (all)	unsub	D1 takers	D1
non-tk							

Full-time employee	3341	33.6%	11.6%	27.1%	11.8%	61.2%	
22.6%							
Business owner	1817	18.3%	8.8%	24.9%	9.4%	59.4%	
21.5%							
Freelancer / Self-employed	1487	14.9%	11.6%	26.1%	10.5%	63.0%	
21.2%							
Exploring options	845	8.5%	12.3%	26.0%	18.6%	55.8%	
21.9%							
Full-time worker	731	7.3%	12.3%	28.5%	28.5%	57.8%	
24.3%							
Between jobs / Career switcher	578	5.8%	13.1%	26.5%	13.8%	61.8%	
21.1%							
Student	175	1.8%	14.9%	20.0%	37.1%	53.8%	
14.1%							
— GOAL —							
segment	subs	% cohort	take rate	D1 (all)	unsub	D1 takers	D1
non-tk							

Work faster	3015	30.3%	10.0%	25.4%	10.9%	60.4%	
21.5%							
Feel more confident with AI	1992	20.0%	10.3%	25.5%	10.7%	62.6%	
21.2%							
Start my own business	1161	11.7%	14.4%	29.8%	10.9%	64.7%	
23.9%							
Earn more	830	8.3%	11.3%	27.7%	10.7%	64.9%	

23.0%						
Get a promotion or a better job	636	6.4%	11.0%	25.0%	14.0%	50.0%
21.9%						
Gain flexibility or work remotely	492	4.9%	13.4%	25.2%	27.4%	57.6%
20.2%						
Get a quick side hustle	368	3.7%	10.6%	21.2%	37.2%	51.3%
17.6%						
Unlock better income opportunitie	347	3.5%	15.3%	31.1%	25.4%	50.9%
27.6%						
Increase work productivity	185	1.9%	10.8%	24.3%	10.3%	60.0%
20.0%						

— COUNTRY —							
segment	subs	% cohort	take rate	D1 (all)	unsub	D1 takers	D1
non-tk							

US	3749	37.7%	9.8%	24.2%	12.5%	60.1%	
20.3%							
GB	1020	10.2%	10.0%	26.6%	12.6%	65.7%	
22.2%							
AU	691	6.9%	10.9%	25.6%	16.4%	60.0%	
21.4%							
CA	523	5.3%	12.4%	24.5%	16.3%	55.4%	
20.1%							
IT	375	3.8%	14.4%	26.1%	17.3%	53.7%	
21.5%							
AE	263	2.6%	14.8%	26.6%	12.2%	41.0%	
24.1%							
DE	255	2.6%	11.4%	26.7%	20.8%	79.3%	
19.9%							
FR	250	2.5%	10.4%	23.6%	14.4%	57.7%	
19.6%							
SG	243	2.4%	12.3%	30.5%	10.7%	56.7%	
26.8%							
ES	229	2.3%	14.8%	24.5%	14.0%	52.9%	
19.5%							
MX	202	2.0%	8.4%	29.2%	5.4%	88.2%	
23.8%							
CO	154	1.5%	10.4%	24.0%	9.1%	56.2%	
20.3%							

— AGE × GENDER (the PM's claim: 'our main market is 40-50 y.o. men') —					
age	Male	Female	other/NA	row total	row %

18-24	602	310	1	913	9.2%
25-34	613	493	11	1117	11.2%
35-44	1040	815	32	1887	19.0%
45-54	1444	1065	42	2551	25.6%
55+	2120	1035	45	3200	32.1%
TOTAL	5819	3718	131	9956	
	58.4%	37.3%			

Men aged 35-54 : 2484 (24.9% of paying subscribers)
 Everyone aged 35-54 : 4438 (44.6%)
 All men : 5819 (58.4%)
 All women : 3718 (37.3%)
 Aged 45+ : 5751 (57.8%)

§7 · UNSUBSCRIBES

total unsubscribed in cohort: 1393 / 9956 = 14.0%

plan	subs	unsub	rate

4Week	6341	773	12.2% [11.4%–13.0%]
12Week	2495	264	10.6% [9.4%–11.8%]
1Week	1014	351	34.6% [31.8%–37.6%]
4Week_special	90	3	3.3% [1.1%–9.3%]

tier	n	unsub	rate
HIGH (3+ lessons)	158	18	11.4% [7.3%–17.3%]
LOW (0–2 lessons)	953	109	11.4% [9.6%–13.6%]
DIDN'T TAKE IT	8845	1266	14.3% [13.6%–15.1%]

Stated reason (only recorded for some unsubsribes):

(none recorded)	873	(62.7% of unsubs)
Support request	310	(22.3% of unsubs)
hard_decline	84	(6.0% of unsubs)
account_deletion	31	(2.2% of unsubs)
Mastercard Alert	30	(2.2% of unsubs)
general	22	(1.6% of unsubs)
dispute	12	(0.9% of unsubs)
paypal_refund	10	(0.7% of unsubs)
Visa CDRN	7	(0.5% of unsubs)
fraud	6	(0.4% of unsubs)
Merchanto visa	4	(0.3% of unsubs)
payment_refunded	3	(0.2% of unsubs)
fraud_reported	1	(0.1% of unsubs)

→ payment-failure / chargeback / refund-dispute reasons: 157 (11.3% of unsubs, 1.6% of the cohort)

Do challenge takers unsubscribe less? (raw, uncontrolled)

takers	127/1111	= 11.4%	[9.7%–13.4%]
non-takers	1266/8845	= 14.3%	[13.6%–15.1%]
diff	-2.9%	z=-2.61	p=0.009 **

Decisions settled before the tiers were chosen

subscribers: 9956 · challenge-338 takers: 1111

Q1 · lag between first app day and challenge start (days)

+ 0 days:	266	(23.9%)	cum	23.9%
+ 1 days:	342	(30.8%)	cum	54.7%
+ 2 days:	196	(17.6%)	cum	72.4%
+ 3 days:	90	(8.1%)	cum	80.5%
+ 4 days:	65	(5.9%)	cum	86.3%
+ 5 days:	58	(5.2%)	cum	91.5%
+ 6 days:	42	(3.8%)	cum	95.3%
+ 7 days:	24	(2.2%)	cum	97.5%
+ 8 days:	15	(1.4%)	cum	98.8%
+ 9 days:	3	(0.3%)	cum	99.1%
+10 days:	6	(0.5%)	cum	99.6%
+11 days:	2	(0.2%)	cum	99.8%
+12 days:	1	(0.1%)	cum	99.9%
+13 days:	1	(0.1%)	cum	100.0%

→ Only 23.9% of takers start the challenge on their FIRST app day;
the median taker starts on day +1, and 28% start on day +3 or later.
CONSEQUENCE: the two anchors are NOT interchangeable, and takers are pre-selected survivors – to start on day k you must have survived to day k (immortal-time bias).
So: use the first-seen anchor for all cross-group comparisons, and treat even that as favourable to takers. See 02_main.py §4 Test B.

Q2 · challenge lessons STARTED vs COMPLETED (the >7 spike)

completed: {0: 499, 1: 304, 2: 150, 3: 53, 4: 25, 5: 18, 6: 10, 7: 5, 8: 47}
started: {0: 1, 1: 698, 2: 234, 3: 60, 4: 32, 5: 19, 6: 12, 7: 6, 8: 49}

users with >7 completions: 47
their completed-course flag: Counter({1: 47})
their certificate flag: Counter({0: 25, 1: 22})
their active_challenge_days: {1: 13, 2: 9, 3: 5, 4: 4, 5: 5, 6: 2, 7: 3, 8: 3, 9: 2, 10: 1}
their lessons_started: {8: 47}

cross-check – completed_course flag by completion count:

0 lessons:	course_completed	0/ 499
1 lessons:	course_completed	0/ 304
2 lessons:	course_completed	0/ 150
3 lessons:	course_completed	0/ 53
4 lessons:	course_completed	0/ 25
5 lessons:	course_completed	0/ 18
6 lessons:	course_completed	0/ 10
7 lessons:	course_completed	1/ 5
8 lessons:	course_completed	47/ 47

Q3 · DOSE-RESPONSE: outcome by exact number of challenge lessons completed
(retention anchored on first_seen so it is comparable across everyone)

lessons	n		D1	D3	D7 (n obs)		unsub
0	499	58.3% [53.9%–62.6%]	33.5% [29.5%–37.7%]	18.5% [15.0%–22.7%]	(383)	9.8% [7.5%–12.7%]	
1	304	59.5% [53.9%–64.9%]	34.5% [29.4%–40.0%]	17.2% [13.0%–22.5%]	(238)	12.5% [9.2%–16.7%]	
2	150	56.7% [48.7%–64.3%]	37.3% [30.0%–45.3%]	18.1% [12.2%–26.1%]	(116)	14.7% [9.9%–21.2%]	
3	53	60.4% [46.9%–72.4%]	35.8% [24.3%–49.3%]	39.0% [25.7%–54.3%]	(41)	13.2% [6.5%–24.8%]	
4	25	60.0% [40.7%–76.6%]	68.0% [48.4%–82.8%]	31.8% [16.4%–52.7%]	(22)	12.0% [4.2%–30.0%]	
5	18	72.2% [49.1%–87.5%]	61.1% [38.6%–79.7%]	38.5% [17.7%–64.5%]	(13)	5.6% [1.0%–25.8%]	
6	10	100.0% [72.2%–100.0%]	80.0% [49.0%–94.3%]	50.0% [21.5%–78.5%]	(8)	10.0% [1.8%–40.4%]	

43.4%]	7	5		80.0% [37.6%-96.4%]	100.0% [56.6%-100.0%]	40.0% [11.8%-76.9%]	(5)		0.0% [0.0%-
	8	47		76.6% [62.8%-86.4%]	63.8% [49.5%-76.0%]	40.9% [27.7%-55.6%]	(44)		12.8% [6.0%-25.2%]
same, by ACTIVE CHALLENGE DAYS (the mechanic's own unit: 1 day = 1 skill)									
	lessons	n		D1	D3	D7 (n obs)			unsub

14.4%]	1	816		53.7% [50.2%-57.1%]	29.4% [26.4%-32.6%]	14.7% [12.1%-17.7%]	(614)		12.0% [10.0%-
	2	163		76.1% [69.0%-82.0%]	46.6% [39.1%-54.3%]	28.6% [21.7%-36.5%]	(140)		11.0% [7.1%-16.8%]
32.4%]	3	59		76.3% [64.0%-85.3%]	74.6% [62.2%-83.9%]	36.0% [24.1%-49.9%]	(50)		8.5% [3.7%-18.4%]
	4	29		79.3% [61.6%-90.2%]	79.3% [61.6%-90.2%]	39.1% [22.2%-59.2%]	(23)		10.3% [3.6%-26.4%]
	5	21		90.5% [71.1%-97.3%]	76.2% [54.9%-89.4%]	52.4% [32.4%-71.7%]	(21)		14.3% [5.0%-34.6%]
	6	8		62.5% [30.6%-86.3%]	100.0% [67.6%-100.0%]	71.4% [35.9%-91.8%]	(7)		0.0% [0.0%-
	7	15		86.7% [62.1%-96.3%]	73.3% [48.0%-89.1%]	80.0% [54.8%-93.0%]	(15)		0.0% [0.0%-20.4%]

```

=====
Q4 · observation window / censoring
=====

last data date: 2026-06-25
days from first_seen to last data date:
  0 days:      3
  1 days:     10
  2 days:      2
  3 days:     10
  4 days:    862
  5 days:    878
  6 days:    976
  7 days:   1167
  8 days:   1270
  9 days:   1051
 10 days:    911
 11 days:   1057
 12 days:    877
 13 days:    882
D1 observable for 9953/9956 (100.0%) of subscribers
D3 observable for 9941/9956 (99.8%) of subscribers
D7 observable for 7215/9956 (72.5%) of subscribers

```

```

=====
Q5 · is challenge 338 effectively THE Challenge feature?
=====

started_any_challenge: 1114
group_338 == took_338: 1111
took_other_challenge : 3
→ 338 covers 99.7% of all challenge starters.

```

Supplementary tests — CSAT, refund gate, data quality

=====

\$A · D1 AS THE TEAM DEFINED IT vs D1 ON A COMMON CLOCK

=====

Brief: 'Target metric: D1 Retention (retention the day after the challenge started).'
That anchors takers on their challenge-start day. Non-takers have no such day, so the only way to compare is a common clock (first app day). Both are shown – the difference is the single biggest reason a naive read of this release looks better than it is.

measure	n	rate
TAKERS · D1 anchored on challenge-start (team defn)	1111	39.8% [36.9%–42.7%]
TAKERS · D1 anchored on first app day	1111	60.0% [57.1%–62.9%]
NON-TAKERS · D1 anchored on first app day	8845	21.7% [20.9%–22.6%]
TAKERS · D3 anchored on challenge-start	982	25.9% [23.2%–28.7%]
TAKERS · D7 anchored on challenge-start	606	14.7% [12.1%–17.7%]

=====

\$B · CSAT – does engaging with the product make people happier?

=====

group	raters	mean CSAT	% 4-5	% 1-2
Completed the whole challenge	48	3.10	37.5%	22.9%
HIGH (3+ challenge lessons)	158	3.33	38.6%	19.6%
LOW (0-2 challenge lessons)	845	3.53	53.0%	20.6%
Never took the challenge	3740	3.74	59.8%	17.5%

CSAT given INSIDE challenge 338 specifically: n=583 raters, mean 3.25
... of 1111 starters, only 52.5% ever rated challenge content.
Same users' CSAT across ALL product content: 3.45

=====

\$C · COMPLETION AND THE REFUND GATE

=====

Part A found (from public reviews) that finishing the plan is the money-back gate.
If true, completion should predict MORE unsubscribing, not less. Test it.

group	n	unsubscribe rate
finished the challenge (all 8 lessons)	48	12.5% [5.9%–24.7%]
started but did not finish	1063	11.4% [9.6%–13.4%]
never took it	8845	14.3% [13.6%–15.1%]

finishers vs non-finishers: diff +1.1% z=+0.24 p=0.812 ns

of the finishers, those who binged it in <=2 days: n=22, unsub 22.7% [10.1%–43.4%]
of the finishers, those who spread it over 3+ days: n=26, unsub 3.8% [0.7%–18.9%]

took the certificate: n=23, unsub 13.0% [4.5%–32.1%]

=====

\$D · THE 'SAW THE POPUP AND WALKED AWAY' GROUP – the worst cell in the dataset

=====

group	n	D1	unsub	avg lessons
saw popup, never opened the challenge	1132	44.9% [42.0%–47.8%]	25.5% [23.1%–28.1%]	3.71
never reached any challenge surface	6716	12.2% [11.4%–13.0%]	12.5% [11.7%–13.3%]	0.81
took the challenge	1111	60.0% [57.1%–62.9%]	11.4% [9.7%–13.4%]	8.22

plan mix of 'saw popup, never opened': [('4Week', 703), ('12Week', 266), ('1Week', 144), ('4Week_special', 15)]
plan mix of the whole cohort : [('4Week', 6341), ('12Week', 2495), ('1Week', 1014), ('4Week_special', 90)]

1-Week subscribers as share of 'saw popup, never opened': 12.7% (cohort: 10.2%)

=====

\$E · DATA-QUALITY FLAGS (things I would raise before anyone acts on this)

=====

1. Two different quiz vocabularies are mixed in one cohort:
age '45-54': n=2551 unsub= 9.5% D1= 26.9%
age '55+': n=3200 unsub= 9.7% D1= 25.4%
age '45+': n=235 unsub= 17.0% D1= 37.0%
status 'Full-time employee': n=3341 unsub= 11.8% D1= 27.1%
status 'Full-time worker': n=731 unsub= 28.5% D1= 28.5%
→ 'Full-time worker' churns at 2.4x 'Full-time employee'. Same words, different funnel.
Either a different quiz version or different traffic. Cannot be pooled blindly.
2. THE BIG ONE – there is no hold-out group. Challenge starts run on every single day of the window (2026-06-12 ... 2026-06-25), so there is no pre/post and no control arm. Every causal statement below is an observational correction, not an experiment. This is the #1 thing to fix before the next release, not after it.
3. Observation window is only 14 days (2026-06-12 → 2026-06-25):
D7 is unobservable for 27.5% of the cohort;
unsubscribe is right-censored for everyone (a 4-week plan cannot even reach its first renewal inside this window), so all unsubscribe rates here are FLOOR values.
4. The popup carries no challenge_id, so 'saw the popup' is exposure to ANY challenge, not specifically 338. 338 is 99.7% of all starts, so the practical impact is small.

=====

\$F · WHAT THE FEATURE IS WORTH TODAY

=====

Reach: 1111/9956 = 11.2% of paying subscribers started it.
Activation: 612/1111 = 55.1% did even one lesson.
Completion: 48/1111 = 4.3% finished (0.48% of all subscribers).

Best causal estimate of the D1 effect (vs the exposure-matched control):
+0.5% percentage points, p=0.823 (ns) – statistically indistinguishable from zero.

Even if the effect were real and equal to the raw taker-vs-non-taker gap, at 11.2% reach the cohort-level D1 impact would be at most 0.112 x the gap.

raw gap = +38.3% → cohort-level ceiling = +4.3% pts of overall D1.

Overall cohort D1 today: 26.0%

Cohort baselines and the stratified control test

```
== cohort-level baselines (all 9,956 paying subscribers, first-app-day anchor) ==
D1: 26.0% [25.2%-26.9%]    (n=9953)
D3: 13.1% [12.5%-13.8%]    (n=9941)
D7: 8.6% [8.0%-9.3%]      (n=7215)
unsubscribed within the 14-day window: 14.0% [13.3%-14.7%]
mean lessons completed (any course): 2.53
mean active days in product: 1.98
never completed a single lesson anywhere: 50.2%
finished onboarding: 69.8%

== CONFOUND CHECK for $D: is 'saw the popup' just a proxy for time-in-product? ==
group                n  mean active days  mean days observed  unsub
took 338              1111          3.73              8.94      11.4%
saw popup, never opened 1132          2.74              8.95      25.5%
never reached a surface 6716          1.33              8.27      12.5%

Same three groups, holding ACTIVE DAYS roughly constant (users with 3-6 active days):
group                n  D1              unsub
took 338              622 69.6% [65.9%-73.1%]  11.3% [9.0%-14.0%]
saw popup, never opened 516 63.4% [59.1%-67.4%]  24.0% [20.5%-27.9%]
never reached a surface 326 58.9% [53.5%-64.1%]  19.3% [15.4%-24.0%]

== TAKERS vs EXPOSED-CONTROL, holding active days constant (the tightest test I can run) ==
active days  n takers  D1 takers  n control  D1 control  diff  p
1            97       0.0%       63       0.0%    +0.0%  1.000
2           270      46.3%      257      42.4%    +3.9%  0.370
3-4          446      64.8%      427      65.1%    -0.3%  0.924
5-7          217      82.9%      205      84.4%    -1.4%  0.689
8+           81      90.1%       42      76.2%   +13.9%  0.038

== was the Challenge live for the whole observation window? ==
2026-06-12: 39 starts
2026-06-13: 69 starts
2026-06-14: 101 starts
2026-06-15: 124 starts
2026-06-16: 70 starts
2026-06-17: 88 starts
2026-06-18: 115 starts
2026-06-19: 102 starts
2026-06-20: 84 starts
2026-06-21: 84 starts
2026-06-22: 106 starts
2026-06-23: 76 starts
2026-06-24: 38 starts
2026-06-25: 15 starts

== plan mix: real cohort vs the plans.csv assumption used in Part D ==
1Week : 10.2% (plans.csv assumes 10%)
4Week : 64.6% (plans.csv assumes 70%)
12Week: 25.1% (plans.csv assumes 20%)
```

Part A — k-modes segment clustering

```
quiz respondents in export          49,487
answered >= 6 of 10 core questions 38,071 (76.9%) <- the clustering sample

== choosing k: within-cluster Hamming cost ==
k      cost    cost/user  improvement
2  193,760      5.089      -
3  182,252      4.787      5.9%
4  177,769      4.669      2.5%
5  171,100      4.494      3.8%
6  167,414      4.397      2.2%
7  164,227      4.314      1.9%
8  159,839      4.198      2.7%

== k = 6 · cluster profiles (lift = share in cluster / share in population) ==

— cluster 0  n = 10,810 (28.4%)
  1.87x 62.3% feels_ready      Somewhat ready – but I could be behind
  1.63x 49.6% learn_claude_for Growth - I love learning in-demand skills
  1.61x 42.7% benefit         Feel more confident with AI
  1.52x 41.8% age             45-54
  1.33x 69.7% ai_experience    Good, but i feel like i'm only scratching the surfac
  1.28x 75.0% gender          Male

— cluster 3  n = 8,372 (22.0%)
  2.22x 50.3% feels_ready      A bit worried – things are changing quickly
  2.17x 61.0% career_fear      Falling behind as others move faster
  1.87x 82.0% used_claude      Not yet
  1.59x 40.4% field            Other
  1.43x 48.9% age             55+
  1.29x 44.6% benefit          Work faster

— cluster 1  n = 7,953 (20.9%)
  2.51x 45.7% feels_ready      Yes - I'm confident in my skills
  2.50x 60.6% ai_experience     Great - AI already helps me a lot
  1.71x 67.2% gender           Female
  1.66x 57.1% benefit          Work faster
  1.49x 80.1% learn_claude_for Work tasks
  1.39x 58.2% career_fear      Nothing - I see AI as an opportunity, not a threat

— cluster 2  n = 4,699 (12.3%)
  2.59x 49.8% age             35-44
  1.49x 51.4% benefit          Work faster
  1.39x 75.0% learn_claude_for Work tasks
  1.31x 76.6% gender           Male
  1.28x 71.5% used_claude      Yes
  1.17x 49.6% work_status      Full-time employee

— cluster 4  n = 3,413 (9.0%)
  3.11x 50.4% benefit          Start my own business
  2.56x 46.5% feels_ready      Not really – I need new skills fast
  2.16x 84.9% gender           Female
  2.07x 52.7% field            Other
  1.99x 60.8% learn_claude_for Growth - I love learning in-demand skills
  1.77x 48.6% age             45-54

— cluster 5  n = 2,824 (7.4%)
  4.42x 49.6% work_status      Exploring options
  2.43x 44.2% feels_ready      Not really – I need new skills fast
  2.32x 27.2% ai_experience     I haven't really tried yet
  2.22x 58.6% benefit          Feel more confident with AI
  2.04x 31.9% learn_claude_for Personal use
  2.01x 68.9% age             55+
```

SQL · 00_explore_schema

```
-- Part C · Step 0 – LEARN THE REAL SCHEMA FIRST (run this before anything else)
-- Project: persona-496908 · Tables: app_events, subscribe_events
-- The queries in 01-04 use ASSUMED column names (flagged with -- ΔADJUST). Run these
-- three first, then find/replace the assumed names with the real ones.

-- 1) What columns exist? (replace <dataset> with the real dataset id shown in the console)
SELECT table_name, column_name, data_type
FROM `persona-496908.<dataset>.INFORMATION_SCHEMA.COLUMNS`
WHERE table_name IN ('app_events', 'subscribe_events')
ORDER BY table_name, ordinal_position;

-- 2) What do app_events rows look like, and what event names exist?
SELECT * FROM `persona-496908.<dataset>.app_events` LIMIT 50;

SELECT event_name, COUNT(*) AS n          -- ΔADJUST: the event-name column may be `event`, `name`, `type`
FROM `persona-496908.<dataset>.app_events`
GROUP BY event_name ORDER BY n DESC LIMIT 100;
-- ↑ Look here for the Challenge events: something like challenge_start / challenge_day_complete /
--   lesson_complete / day_open. Note the exact strings – 01-04 depend on them.

-- 3) What does subscribe_events carry? (need: user id, subscription start, cancel/unsubscribe, plan, price)
SELECT * FROM `persona-496908.<dataset>.subscribe_events` LIMIT 50;
```

SQL · 01_engagement_tiers

```
-- Part C · Step 1 – Classify each Challenge-eligible user into an engagement tier.
-- ΔADJUST all names marked below to the real schema found in 00_explore_schema.sql.
-- Tiers:  high = started AND (reached >=Day3 OR returned on >=3 of first 7 days)
--         low  = started but stalled before Day 3
--         none = eligible but never opened Day 1

DECLARE challenge_id STRING DEFAULT '338';          -- the Challenge in the brief (skills/challenges/338)

WITH ev AS (
  SELECT
    user_id,                                -- ΔADJUST
    event_name,                             -- ΔADJUST
    TIMESTAMP(event_timestamp) AS ts,        -- ΔADJUST
    DATE(TIMESTAMP(event_timestamp)) AS d,
    SAFE_CAST(JSON_VALUE(event_params, '$.challenge_id') AS STRING) AS ch -- ΔADJUST (may be a column)
  FROM `persona-496908.<dataset>.app_events`
),
challenge AS (                                -- events belonging to THIS challenge
  SELECT * FROM ev WHERE ch = challenge_id OR ch IS NULL -- drop the OR NULL once the field is confirmed
),
per_user AS (
  SELECT
    user_id,
    MIN(IF(event_name='challenge_start', ts, NULL)) AS started_at,      -- ΔADJUST event
    COUNTIF(event_name='challenge_day_complete') AS days_completed,    -- ΔADJUST
    event
  COUNT(DISTINCT IF(event_name='challenge_day_complete', d, NULL)) AS distinct_active_days,
    MIN(d) AS first_day, MAX(d) AS last_day
  FROM challenge
  GROUP BY user_id
)
SELECT
  user_id,
  started_at IS NOT NULL AS started,
  days_completed,
  distinct_active_days,
  CASE
    WHEN started_at IS NULL THEN 'none'
    WHEN days_completed >= 3 OR distinct_active_days >= 3 THEN 'high'
    ELSE 'low'
  END AS engagement_tier
FROM per_user;

-- Sanity: the tier distribution (use this histogram to calibrate the Day-3 threshold)
-- SELECT engagement_tier, COUNT(*) FROM ( <the query above> ) GROUP BY 1;
```

SQL · 02_core_metrics

```
-- Part C · Step 2 – The core table: target + secondary metrics BY engagement tier.
-- Depends on the tiering in 01. ΔADJUST names to the real schema.
-- D1/D3/D7 retention are measured relative to each user's challenge-start day.

DECLARE challenge_id STRING DEFAULT '338';

WITH tiers AS (
  -- paste/inline the SELECT from 01_engagement_tiers.sql, or save it as a view and reference it
  SELECT * FROM `persona-496908.<dataset>.v_challenge_tiers`      -- Δ create this view from 01
),
starts AS (
  SELECT user_id, DATE(MIN(TIMESTAMP(event_timestamp))) AS d0      -- ΔADJUST
  FROM `persona-496908.<dataset>.app_events`
  WHERE event_name='challenge_start'                                -- ΔADJUST
  GROUP BY user_id
),
activity AS (
  SELECT DISTINCT user_id, DATE(TIMESTAMP(event_timestamp)) AS d -- any app activity = "retained that day"
  FROM `persona-496908.<dataset>.app_events`
),
ret AS (
  SELECT s.user_id,
    MAX(a.d = DATE_ADD(s.d0, INTERVAL 1 DAY)) AS d1,
    MAX(a.d = DATE_ADD(s.d0, INTERVAL 3 DAY)) AS d3,
    MAX(a.d = DATE_ADD(s.d0, INTERVAL 7 DAY)) AS d7
  FROM starts s LEFT JOIN activity a USING(user_id)
  GROUP BY s.user_id
),
unsub AS (
  SELECT user_id, MAX(is_cancelled) AS unsubscribed              -- ΔADJUST: from subscribe_events
  FROM `persona-496908.<dataset>.subscribe_events`                -- e.g. status='cancelled' / cancel_at IS
  NOT NULL
  GROUP BY user_id
)
SELECT
  t.engagement_tier,
  COUNT(*)                                AS users,
  ROUND(AVG(CAST(r.d1 AS INT64)),3)        AS d1_retention,    -- TARGET METRIC
  ROUND(AVG(CAST(r.d3 AS INT64)),3)        AS d3_retention,
  ROUND(AVG(CAST(r.d7 AS INT64)),3)        AS d7_retention,
  ROUND(AVG(CAST(COALESCE(u.unsubscribed,false) AS INT64)),3) AS unsubscribe_rate,
  ROUND(AVG(t.days_completed)/7.0,3)       AS lesson_completion -- of the 7 days
FROM tiers t
LEFT JOIN ret r USING(user_id)
LEFT JOIN unsub u USING(user_id)
GROUP BY t.engagement_tier
ORDER BY CASE t.engagement_tier WHEN 'high' THEN 1 WHEN 'low' THEN 2 ELSE 3 END;

-- READ IT LIKE THIS: the release "worked" only if there is a smooth dose-response
-- (high < low < none on unsubscribe; high > low > none on D1/D3/D7) that survives the
-- baseline/hold-out check in 03. A high-vs-none gap alone can be pure selection.
```

SQL · 03_segment_breakdown

```
-- Part C · Step 3 – Same metrics, cut by segment (age / profession / plan) + the selection guard.
-- ΔADJUST names. Quiz answers (age, status/profession, goal) live on the user/subscribe record or
-- in early app_events params – confirm where in 00.

-- 3a) Metrics by demographic segment × engagement tier
WITH base AS (
  SELECT t.user_id, t.engagement_tier,
         s.age, s.profession, s.plan, -- ΔADJUST source of quiz demographics
         r.d1, r.unsubscribed, t.days_completed
  FROM `persona-496908.<dataset>.v_challenge_metrics_user` t -- Δ a per-user view joining 01+02
  LEFT JOIN `persona-496908.<dataset>.subscribe_events` s USING(user_id)
  LEFT JOIN `persona-496908.<dataset>.v_challenge_ret` r USING(user_id)
)
SELECT age, plan, engagement_tier,
       COUNT(*) users,
       ROUND(AVG(CAST(d1 AS INT64)),3) d1_retention,
       ROUND(AVG(CAST(unsubscribed AS INT64)),3) unsubscribe_rate
FROM base
GROUP BY age, plan, engagement_tier
ORDER BY age, plan, engagement_tier;

-- 3b) THE SELECTION GUARD – dose-response: does each EXTRA challenge day predict lower unsubscribe?
-- If the slope is smooth and monotonic, the effect is more plausibly causal than pure self-selection.
SELECT days_completed,
       COUNT(*) users,
       ROUND(AVG(CAST(unsubscribed AS INT64)),3) unsubscribe_rate,
       ROUND(AVG(CAST(d1 AS INT64)),3) d1_retention
FROM `persona-496908.<dataset>.v_challenge_metrics_user` -- Δ per-user view
GROUP BY days_completed
ORDER BY days_completed;

-- 3c) IDEAL (if a hold-out or pre-launch cohort exists): compare Challenge-exposed vs not,
-- matched on entry motivation (quiz time_goal / goal). If no hold-out exists, the #1
-- recommendation for the next release is to ship the Challenge as a proper A/B, not 100%.
```

SQL · 10_explore_events

```
-- Stage 1 · probe queries – run these next, paste results back.
-- They tell me the full event list (incl. unsubscribe + any challenge-complete/streak events)
-- and exactly how Challenge participation is logged, so I can build the real tier metrics.

-- Q1 – every event type + volume in app_events
SELECT event_name, COUNT(*) AS n
FROM `persona-496908.sql_assessment.app_events`
GROUP BY event_name
ORDER BY n DESC;

-- Q2 – how is the Challenge logged? (path vs course vs category)
SELECT event_name,
       REGEXP_EXTRACT(path, r'/skills/challenges/(\d+)') AS challenge_id,
       course_id, course_name, category_name, mechanic_name,
       COUNT(*) AS n
FROM `persona-496908.sql_assessment.app_events`
WHERE path LIKE '/skills/challenges/%'
      OR LOWER(event_name) LIKE '%challenge%'
      OR LOWER(IFNULL(category_name, '')) LIKE '%challenge%'
GROUP BY 1,2,3,4,5,6
ORDER BY n DESC
LIMIT 200;

-- Q3 – do challenge lessons carry a distinct course/category? peek at raw challenge rows
SELECT event_name, user_id, timestamp, path, course_id, course_name,
       lesson_id, lesson_name, lesson_order, category_name, mechanic_name, status
FROM `persona-496908.sql_assessment.app_events`
WHERE path LIKE '/skills/challenges/%' OR LOWER(event_name) LIKE '%challenge%'
ORDER BY timestamp
LIMIT 50;

-- Q4 – the unsubscribe event (which event_name carries unsubscribe_reason?)
SELECT event_name, unsubscribe_reason, COUNT(*) AS n
FROM `persona-496908.sql_assessment.app_events`
WHERE unsubscribe_reason IS NOT NULL
GROUP BY 1,2
ORDER BY n DESC
LIMIT 100;
```

SQL · 11_challenge_funnel

```
-- Stage 1 · Part C – first real query. Run this, paste the one-row result back.  
-- A compact engagement funnel using the confirmed event names (Events Convention).  
-- It returns distinct-user counts so we can see the shape before building tier metrics.
```

```
SELECT  
    COUNT(DISTINCT IF(event_name='pr_webapp_launch_first_time',      user_id, NULL)) AS registered,  
    COUNT(DISTINCT IF(event_name='pr_webapp_challenge_view',        user_id, NULL)) AS  
challenge_viewed,  
    COUNT(DISTINCT IF(event_name='pr_webapp_challenge_start',      user_id, NULL)) AS  
challenge_started,  
    COUNT(DISTINCT IF(event_name='pr_webapp_challenge_join',        user_id, NULL)) AS  
challenge_joined,  
    COUNT(DISTINCT IF(event_name='pr_webapp_lesson_completed',      user_id, NULL)) AS  
completed_any_lesson,  
    COUNT(DISTINCT IF(event_name='pr_webapp_challenge_certificate_download',user_id, NULL)) AS  
got_challenge_certificate,  
    COUNT(DISTINCT IF(event_name='pr_webapp_challenge_csat_click',  user_id, NULL)) AS  
gave_challenge_csat,  
    COUNT(DISTINCT IF(event_name='pr_webapp_unsubscribed',          user_id, NULL)) AS unsubscribed,  
    COUNT(DISTINCT user_id) AS all_users,  
    MIN(timestamp) AS first_event, MAX(timestamp) AS last_event  
FROM `persona-496908.sql_assessment.app_events`;
```

```
-- After this: I'll write query 12 = engagement tiers (high/low/didn't-start) and the  
-- D1/D3/D7 + unsubscribe + completion + CSAT comparison, per your definitions.
```


SQL · 12_challenge338_user_table

```
-- 12 · Per-user table for the 7-Day Claude Challenge (id 338).
-- RUN this, then "Save results ► CSV" and drop the file in part-c-release-verdict/data/.
-- One row per user who STARTED challenge 338. I'll do the tier / D1-D3-D7 / segment analysis locally.
-- (If it throws an error, paste me the error text – I can't test against your DB.)

WITH ch338 AS (
    -- every event belonging to challenge 338
    SELECT user_id, event_name, timestamp, DATE(timestamp) AS d, csat_score
    FROM `persona-496908.sql_assessment.app_events`
    WHERE course_id = 338 OR REGEXP_CONTAINS(IFNULL(path,''), r'challenges/338')
),
starter AS (
    -- per-user challenge-338 engagement + outcomes
    SELECT
        user_id,
        MIN(IF(event_name='pr_webapp_challenge_start', d, NULL)) AS start_date,
        COUNT(DISTINCT d) AS active_challenge_days,
        COUNTIF(event_name='pr_webapp_lesson_completed') AS lessons_completed,
        MAX(IF(event_name='pr_webapp_course_completed',1,0)) AS completed_course,
        MAX(IF(event_name IN ('pr_webapp_challenge_certificate_download',
            'pr_webapp_challenge_certificate_share'),1,0)) AS got_certificate,
        COUNTIF(event_name IN ('pr_webapp_lesson_csat_click','pr_webapp_challenge_csat_click')) AS csat_n,
        AVG(IF(event_name IN ('pr_webapp_lesson_csat_click','pr_webapp_challenge_csat_click'),
            csat_score, NULL)) AS avg_csat
    FROM ch338
    GROUP BY user_id
),
act AS ( SELECT user_id, DATE(timestamp) AS d
    FROM `persona-496908.sql_assessment.app_events` GROUP BY 1,2 ),
uns AS ( SELECT user_id, ANY_VALUE(unsubscribe_reason) AS unsub_reason
    FROM `persona-496908.sql_assessment.app_events`
    WHERE event_name='pr_webapp_unsubscribed' GROUP BY 1 ),
seg AS ( SELECT user_id, ANY_VALUE(age) age, ANY_VALUE(gender) gender, ANY_VALUE(goal) goal,
    ANY_VALUE(status) status, ANY_VALUE(subscription) plan
    FROM `persona-496908.sql_assessment.subscribe_events` GROUP BY 1 )
SELECT
    s.user_id, s.start_date, s.active_challenge_days, s.lessons_completed,
    s.completed_course, s.got_certificate, s.csat_n, ROUND(s.avg_csat,2) AS avg_csat,
    MAX(IF(a.d = DATE_ADD(s.start_date, INTERVAL 1 DAY),1,0)) AS ret_d1,
    MAX(IF(a.d = DATE_ADD(s.start_date, INTERVAL 3 DAY),1,0)) AS ret_d3,
    MAX(IF(a.d = DATE_ADD(s.start_date, INTERVAL 7 DAY),1,0)) AS ret_d7,
    IF(u.user_id IS NULL, 0, 1) AS unsubscribed, u.unsub_reason,
    g.age, g.gender, g.goal, g.status, g.plan
FROM starter s
LEFT JOIN act a ON a.user_id = s.user_id
LEFT JOIN uns u ON u.user_id = s.user_id
LEFT JOIN seg g ON g.user_id = s.user_id
WHERE s.start_date IS NOT NULL
-- only users who actually STARTED the challenge
GROUP BY s.user_id, s.start_date, s.active_challenge_days, s.lessons_completed,
    s.completed_course, s.got_certificate, s.csat_n, s.avg_csat,
    u.user_id, u.unsub_reason, g.age, g.gender, g.goal, g.status, g.plan
ORDER BY s.start_date;
```

SQL · 13_comparison_groups

```
-- 13 · THE COMPARISON QUERY – one row per user, takers AND non-takers of Challenge 338.
-- This is the missing third of Part C Task 1: "high / low / didn't take it".
-- Run it once, then "SAVE RESULTS ► CSV (local file)" into part-c-release-verdict/data/.
-- ~10,000 rows (small enough for the local-file download). All tiering + stats done locally in Python.
-- If it errors, paste me the error text verbatim.

WITH
bounds AS (
    -- last day present in the data (for window correction)
    SELECT MAX(DATE(timestamp)) AS last_data_date
    FROM `persona-496908.sql_assessment.app_events`
),
ev AS (
    -- every app event + a flag for "belongs to challenge 338"
    SELECT
        user_id,
        event_name,
        DATE(timestamp) AS d,
        csat_score,
        unsubscribe_reason,
        country_code,
        (course_id = 338 OR REGEXP_CONTAINS(IFNULL(path, ''), r'challenges/338')) AS ch338
    FROM `persona-496908.sql_assessment.app_events`
),
per_user AS (
    -- one row per user: engagement, outcomes, challenge-338
    detail
    SELECT
        user_id,
        MIN(d) AS first_seen,
        COUNT(DISTINCT d) AS active_days_total,
        ANY_VALUE(country_code) AS country_code,

        -- overall product engagement (all courses, not just the challenge)
        COUNTIF(event_name = 'pr_webapp_lesson_completed') AS lessons_completed_all,
        MAX(IF(event_name = 'pr_webapp_onboarding_finished', 1, 0)) AS finished_onboarding,
        MAX(IF(event_name = 'pr_webapp_personal_plan_started', 1, 0)) AS started_personal_plan,

        -- outcome metrics
        MAX(IF(event_name = 'pr_webapp_unsubscribed', 1, 0)) AS unsubscribed,
        ANY_VALUE(IF(event_name = 'pr_webapp_unsubscribed', unsubscribe_reason, NULL)) AS unsub_reason,
        COUNTIF(event_name IN ('pr_webapp_lesson_csat_click',
                                'pr_webapp_module_csat_click',
                                'pr_webapp_challenge_csat_click')) AS csat_n_all,
        AVG(IF(event_name IN ('pr_webapp_lesson_csat_click',
                                'pr_webapp_module_csat_click',
                                'pr_webapp_challenge_csat_click'), csat_score, NULL)) AS avg_csat_all,

        -- exposure to the challenge surface (any challenge – the popup carries no course_id)
        MAX(IF(event_name = 'pr_webapp_challenge_popup_view', 1, 0)) AS saw_challenge_popup,
        MAX(IF(event_name = 'pr_webapp_challenge_popup_click', 1, 0)) AS clicked_challenge_popup,
        MAX(IF(event_name = 'pr_webapp_challenge_view', 1, 0)) AS viewed_any_challenge,
        MAX(IF(event_name = 'pr_webapp_challenge_start', 1, 0)) AS started_any_challenge,

        -- challenge 338 specifically
        MIN(IF(ch338 AND event_name = 'pr_webapp_challenge_start', d, NULL)) AS ch_start_date,
        MAX(IF(ch338 AND event_name = 'pr_webapp_challenge_view', 1, 0)) AS ch_viewed,
        MAX(IF(ch338 AND event_name = 'pr_webapp_challenge_join', 1, 0)) AS ch_joined,
        COUNTIF(ch338 AND event_name = 'pr_webapp_lesson_completed') AS ch_lessons_completed,
        COUNTIF(ch338 AND event_name = 'pr_webapp_lesson_started') AS ch_lessons_started,
        MAX(IF(ch338 AND event_name = 'pr_webapp_course_completed', 1, 0)) AS ch_completed_course,
        MAX(IF(ch338 AND event_name IN ('pr_webapp_challenge_certificate_download',
                                         'pr_webapp_challenge_certificate_share'), 1, 0)) AS ch_certificate,
        COUNTIF(ch338 AND event_name IN ('pr_webapp_lesson_csat_click',
                                         'pr_webapp_challenge_csat_click')) AS ch_csat_n,
```

```

        AVG(IF(ch338 AND event_name IN ('pr_webapp_lesson_csat_click',
                                         'pr_webapp_challenge_csat_click'), csat_score, NULL)) AS ch_avg_csat
    FROM ev
    GROUP BY user_id
),

anchored AS (
    -- day 0 for retention: challenge start for takers, first app
    day for everyone else
    SELECT
        p.*,
        COALESCE(p.ch_start_date, p.first_seen) AS anchor_date
    FROM per_user p
),

days AS (
    -- distinct active days per user
    SELECT DISTINCT user_id, DATE(timestamp) AS d
    FROM `persona-496908.sql_assessment.app_events`
),

ret AS (
    -- retention off BOTH anchors, so takers and non-takers are
    comparable
    SELECT
        a.user_id,
        MAX(IF(dd.d = DATE_ADD(a.anchor_date, INTERVAL 1 DAY), 1, 0)) AS ret_d1,
        MAX(IF(dd.d = DATE_ADD(a.anchor_date, INTERVAL 3 DAY), 1, 0)) AS ret_d3,
        MAX(IF(dd.d = DATE_ADD(a.anchor_date, INTERVAL 7 DAY), 1, 0)) AS ret_d7,
        MAX(IF(dd.d = DATE_ADD(a.first_seen, INTERVAL 1 DAY), 1, 0)) AS fs_ret_d1,
        MAX(IF(dd.d = DATE_ADD(a.first_seen, INTERVAL 3 DAY), 1, 0)) AS fs_ret_d3,
        MAX(IF(dd.d = DATE_ADD(a.first_seen, INTERVAL 7 DAY), 1, 0)) AS fs_ret_d7
    FROM anchored a
    JOIN days dd USING (user_id)
    GROUP BY a.user_id
),

seg AS (
    -- buyer segmentation (NULL = no purchase row for this user)
    SELECT
        user_id,
        ANY_VALUE(age) AS age,
        ANY_VALUE(gender) AS gender,
        ANY_VALUE(goal) AS goal,
        ANY_VALUE(status) AS status,
        ANY_VALUE(subscription) AS plan,
        MIN(DATE(timestamp)) AS subscribe_date
    FROM `persona-496908.sql_assessment.subscribe_events`
    GROUP BY user_id
)

SELECT
    a.user_id,

    -- the comparison group (I tier "high vs low" locally from ch_lessons_completed)
    CASE
        WHEN a.ch_start_date IS NOT NULL THEN 'took_338'
        WHEN a.ch_joined = 1 THEN 'joined_338_never_started'
        WHEN a.ch_viewed = 1 THEN 'viewed_338_never_joined'
        WHEN a.started_any_challenge = 1 THEN 'took_other_challenge'
        ELSE 'no_challenge'
    END AS group_338,

    a.first_seen,
    a.anchor_date,
    DATE_DIFF(b.last_data_date, a.anchor_date, DAY) AS days_observed, -- <7 ⇒ D7 not observable

    a.ch_start_date, a.ch_lessons_started, a.ch_lessons_completed,
    a.ch_completed_course, a.ch_certificate, a.ch_csat_n, ROUND(a.ch_avg_csat, 2) AS ch_avg_csat,

    a.lessons_completed_all, a.active_days_total,
    a.finished_onboarding, a.started_personal_plan,
    a.saw_challenge_popup, a.clicked_challenge_popup,

```

```
a.viewed_any_challenge, a.started_any_challenge,

r.ret_d1, r.ret_d3, r.ret_d7,
r.fs_ret_d1, r.fs_ret_d3, r.fs_ret_d7,

a.unsubscribed, a.unsub_reason,
a.csat_n_all, ROUND(a.avg_csat_all, 2) AS avg_csat_all,

s.age, s.gender, s.goal, s.status, s.plan, s.subscribe_date,
IF(s.user_id IS NULL, 0, 1) AS has_subscription,
a.country_code
FROM anchored a
CROSS JOIN bounds b
LEFT JOIN ret r ON r.user_id = a.user_id
LEFT JOIN seg s ON s.user_id = a.user_id
ORDER BY a.user_id;
```

Code · Verification — re-derives every published figure

```
"""QA: re-derive every load-bearing number in the write-ups straight from the CSVs and assert it.

If a number in the markdown ever drifts from the data, this fails loudly.
Run: python3 05_qa.py
"""

import csv
import os
from collections import Counter
from lib import load, wilson, two_prop_z, DATA

rows = load()
N = len(rows)
takers = [r for r in rows if r["took"]]
nont = [r for r in rows if not r["took"]]
fails, checks = [], 0

def check(label, actual, expected, tol=0.0005):
    # NB tolerances are deliberately tight: a 0.001 tolerance once hid a 59.557 -> '59.5%' mis-rounding.
    """expected may be an int/float; tol is absolute."""
    global checks
    checks += 1
    ok = abs(actual - expected) <= tol
    if not ok:
        fails.append(f"{label}: computed {actual!r}, document says {expected!r}")
    print(f"  {'ok' if ok else 'FAIL'} {label:<58} {actual:.4f} (doc: {expected})")

def rate(g, k):
    return sum(r[k] for r in g) / len(g)

print("== cohort shape ==")
check("subscribers in cohort", N, 9956, 0)
check("challenge-338 starters", len(takers), 1111, 0)
check("non-takers", len(nont), 8845, 0)
check("all rows have a subscription", sum(r["has_subscription"] for r in rows), 9956, 0)

print("\n== exposure ladder (Part C §1) ==")
check("saw popup", sum(r["saw_challenge_popup"] for r in rows), 3140, 0)
check("saw popup %", sum(r["saw_challenge_popup"] for r in rows) / N, 0.315, 0.0005)
check("clicked popup", sum(r["clicked_challenge_popup"] for r in rows), 2970, 0)
check("viewed a challenge", sum(r["viewed_any_challenge"] for r in rows), 2109, 0)
check("started 338 %", len(takers) / N, 0.112, 0.0005)
check("never reached a surface",
      sum(1 for r in rows if not r["saw_challenge_popup"] and not r["viewed_any_challenge"]), 6716, 0)
check("never reached a surface %",
      sum(1 for r in rows if not r["saw_challenge_popup"] and not r["viewed_any_challenge"]) / N, 0.675,
      0.0005)

print("\n== inside the challenge (Part C §2) ==")
for k, exp in [(1, 612), (2, 308), (3, 158), (8, 47)]:
    check(f"completed >= {k} lessons", sum(1 for r in takers if r["ch_lessons_completed"] >= k), exp, 0)
check("zero lessons %", sum(1 for r in takers if r["ch_lessons_completed"] == 0) / len(takers), 0.449,
      0.0005)
check("lesson1->2 survival", 308 / 612, 0.503, 0.001)
check("completed the course", sum(r["ch_completed_course"] for r in takers), 48, 0)
check("certificate", sum(r["ch_certificate"] for r in takers), 23, 0)

print("\n== tiers + headline table (Part C §4) ==")
high = [r for r in takers if r["ch_lessons_completed"] >= 3]
low = [r for r in takers if r["ch_lessons_completed"] < 3]
```

```

check("HIGH n", len(high), 158, 0)
check("LOW n", len(low), 953, 0)
check("HIGH D1", rate(high, "fs_ret_d1"), 0.696, 0.001)
check("LOW D1", rate(low, "fs_ret_d1"), 0.584, 0.001)
check("DIDNT-TAKE D1", rate(nont, "fs_ret_d1"), 0.217, 0.001)
check("HIGH D3", rate(high, "fs_ret_d3"), 0.570, 0.001)
check("LOW D3", rate(low, "fs_ret_d3"), 0.344, 0.001)
check("DIDNT-TAKE D3", rate(nont, "fs_ret_d3"), 0.100, 0.001)
for label, g, exp, expn in [("HIGH", high, 0.391, 133), ("LOW", low, 0.180, 737), ("DIDNT-TAKE", nont,
0.069, 6345)]:
    obs = [r for r in g if r["fs_days_observed"] >= 7]
    check(f"{label} D7", rate(obs, "fs_ret_d7"), exp, 0.001)
    check(f"{label} D7 denominator", len(obs), expn, 0)
check("HIGH unsub", rate(high, "unsubscribed"), 0.114, 0.001)
check("LOW unsub", rate(low, "unsubscribed"), 0.114, 0.001)
check("DIDNT-TAKE unsub", rate(nont, "unsubscribed"), 0.143, 0.001)

print("\n== the decisive test (Part C §5 Test A) ==")
ctrl = [r for r in rows if r["group_338"] in ("joined_338_never_started", "viewed_338_never_joined")]
check("takers D1", rate(takers, "fs_ret_d1"), 0.600, 0.001)
check("control n", len(ctrl), 994, 0)
check("control D1 (the 59.6%)", rate(ctrl, "fs_ret_d1"), 0.596, 0.0005)
d, z, p = two_prop_z(sum(r["fs_ret_d1"] for r in takers), len(takers),
                      sum(r["fs_ret_d1"] for r in ctrl), len(ctrl))
check("D1 effect (pts)", d, 0.005, 0.001)
check("D1 effect p-value", p, 0.823, 0.005)
d2, _, p2 = two_prop_z(sum(r["unsubscribed"] for r in takers), len(takers),
                      sum(r["unsubscribed"] for r in ctrl), len(ctrl))
check("unsub effect (pts)", d2, -0.023, 0.001)
check("unsub effect p-value", p2, 0.119, 0.005)
check("joined-never-started D1", rate([r for r in rows if r["group_338"] == "joined_338_never_started"],
"fs_ret_d1"), 0.604, 0.001)
check("viewed-never-joined D1", rate([r for r in rows if r["group_338"] == "viewed_338_never_joined"],
"fs_ret_d1"), 0.593, 0.001)

print("\n== immortal time + dose-response (Test B / C) ==")
lag = Counter(r["days_to_challenge"] for r in takers)
check("start on day 0 %", lag[0] / len(takers), 0.239, 0.001)
oneday = [r for r in takers if r["active_challenge_days"] == 1]
hi = [r for r in oneday if r["ch_lessons_completed"] >= 2]
lo = [r for r in oneday if r["ch_lessons_completed"] <= 1]
d3, _, p3 = two_prop_z(sum(r["fs_ret_d1"] for r in hi), len(hi),
                      sum(r["fs_ret_d1"] for r in lo), len(lo))
check("day-0 dose effect (pts, negative)", d3, -0.092, 0.001)
check("day-0 dose p-value", p3, 0.048, 0.002)

print("\n== the mechanic (Test D) ==")
fin = [r for r in takers if r["ch_completed_course"] == 1]
check("finished in 1 day", sum(1 for r in fin if r["active_challenge_days"] == 1), 13, 0)
check("finished in 1 day %", sum(1 for r in fin if r["active_challenge_days"] == 1) / len(fin), 0.271,
0.001)
check("finished in <=2 days %", sum(1 for r in fin if r["active_challenge_days"] <= 2) / len(fin), 0.458,
0.001)
did = [r for r in takers if r["ch_lessons_completed"] >= 1]
check("returned for a 2nd challenge day %",
      sum(1 for r in did if r["active_challenge_days"] >= 2) / len(did), 0.368, 0.001)
check("mean lessons per active challenge day",
      sum(r["ch_lessons_completed"] / max(1, r["active_challenge_days"]) for r in did) / len(did), 1.46,
0.005)

print("\n== the metric that would have fooled the team (§6) ==")
check("takers D1, challenge-anchored", rate(takers, "ret_d1"), 0.398, 0.001)
check("cohort D1, first-app-day", rate([r for r in rows if r["fs_days_observed"] >= 1], "fs_ret_d1"), 0.260,
0.001)

print("\n== CSAT inversion (§8) ==")

def csat(g):

```

```

v = [r["avg_csat_all"] for r in g if r["csat_n_all"] > 0]
return sum(v) / len(v), len(v)

for label, g, exp in [("never took", nont, 3.74), ("LOW", low, 3.53), ("HIGH", high, 3.33), ("finishers",
fin, 3.10)]:
    m, n = csat(g)
    check(f"CSAT {label}", m, exp, 0.005)
chr_ = [r for r in takers if r["ch_csat_n"] > 0]
check("CSAT of challenge-338 content", sum(r["ch_avg_csat"] for r in chr_) / len(chr_), 3.25, 0.005)
check("same users, all content", sum(r["avg_csat_all"] for r in chr_ if r["csat_n_all"] > 0) /
    len([r for r in chr_ if r["csat_n_all"] > 0]), 3.45, 0.005)

print("\n== product-wide baselines (the thesis number) ==")
check("never completed a lesson anywhere %",
    sum(1 for r in rows if r["lessons_completed_all"] == 0) / N, 0.502, 0.001)
check("cohort unsubscribe %", rate(rows, "unsubscribed"), 0.140, 0.001)

print("\n== demographics (Part A + the PM's claim) ==")
check("male %", sum(1 for r in rows if r["gender"] == "Male") / N, 0.597, 0.001)
check("female %", sum(1 for r in rows if r["gender"] == "Female") / N, 0.384, 0.001)
check("aged 45+ (clean buckets) %", sum(1 for r in rows if r["age"] in ("45-54", "55+")) / N, 0.578, 0.001)
check("aged 45+ incl. legacy bucket %",
    sum(1 for r in rows if r["age"] in ("45-54", "55+", "45+")) / N, 0.601, 0.001)
check("men 55+ count", sum(1 for r in rows if r["gender"] == "Male" and r["age"] == "55+"), 2120, 0)
check("men 55+ %", sum(1 for r in rows if r["gender"] == "Male" and r["age"] == "55+") / N, 0.213, 0.001)
check("men 35-54 %", sum(1 for r in rows if r["gender"] == "Male" and r["age"] in ("35-44", "45-54")) / N,
    0.249, 0.001)
for a, exp in [("18-24", 0.311), ("25-34", 0.197), ("35-44", 0.145), ("45-54", 0.095), ("55+", 0.097)]:
    check(f"unsub, age {a}", rate([r for r in rows if r["age"] == a], "unsubscribed"), exp, 0.001)
for gl, exp in [("Work faster", 0.109), ("Feel more confident with AI", 0.107), ("Get a quick side hustle",
0.372)]:
    check(f"unsub, goal '{gl}'", rate([r for r in rows if r["goal"] == gl], "unsubscribed"), exp, 0.001)

print("\n== plans (Part D validation) ==")
pc = Counter(r["plan"] for r in rows)
check("1Week share", pc["1Week"] / N, 0.102, 0.001)
check("4Week share incl. special", (pc["4Week"] + pc["4Week_special"]) / N, 0.646, 0.001)
check("12Week share", pc["12Week"] / N, 0.251, 0.001)
check("1Week unsub", rate([r for r in rows if r["plan"] == "1Week"], "unsubscribed"), 0.346, 0.001)
check("4Week unsub", rate([r for r in rows if r["plan"] == "4Week"], "unsubscribed"), 0.122, 0.001)
check("12Week unsub", rate([r for r in rows if r["plan"] == "12Week"], "unsubscribed"), 0.106, 0.001)
blended = (0.102 * 60.42 + 0.646 * 123.70 + 0.251 * 162.15) / 0.999
check("observed-mix blended net LTV", blended, 126.90, 0.02)

print("\n== unsubscribe reasons (§8) ==")
uns = [r for r in rows if r["unsubscribed"]]
check("total unsubscribed", len(uns), 1393, 0)
invol = {"hard_decline", "Mastercard Alert", "dispute", "paypal_refund", "Visa CDRN",
    "fraud", "Merchanto visa", "payment_refunded", "fraud_reported"}
ni = sum(1 for r in uns if r["unsub_reason"] in invol)
check("payment/chargeback/dispute", ni, 157, 0)
check("... as % of unsubs", ni / len(uns), 0.113, 0.001)

print("\n== event catalogue (the 'not observable here' claim) ==")
# The verdict rests on primary metrics being uncomputable from this dataset. That is a claim
# about the event catalogue, not about the user table, so it needs its own check or it would
# be the one published number nothing re-derives.
CAT = os.path.join(DATA, "script_job_ee3720964a3e4c3ac51fe64306b65890_0.csv")
cat = {r["event_name"]: int(r["n"]) for r in csv.DictReader(open(CAT))}
check("event catalogue rows (file integrity)", len(cat), 67, 0)
check("subscription_renewed events", cat.get("pr_webapp_subscription_renewed", 0), 17, 0)
check("... per subscriber", cat.get("pr_webapp_subscription_renewed", 0) / N, 0.0017, 0.0001)

print("\n" + "=" * 70)
if fails:
    print(f"{len(fails)} of {checks} CHECKS FAILED:")
    for f in fails:
        print("    -", f)

```

```
    raise SystemExit(1)
print(f"All {checks} checks passed – every documented number matches the data.")
```


Code · Part D — the LTV and A/B model

```
#!/usr/bin/env python3
"""
Part D – Subscription economics model (Jobscape PM take-home).

Task 1: Total Net LTV over a one-year horizon (per plan gross+net, blended).
Task 2: A/B plan-upgrade break-even (4-week -> 12-week upgrade replacing 2nd upsell).

Design decisions / assumptions (documented explicitly, per brief):
- Cohort of 1 subscriber per plan; everyone pays the intro (survival S0 = 1).
- Cmn = P(pay period n | survived to period m). Survival S_k = product of C01..C_{k-1,k}.
- Period 0 = intro (pays Intro Price). Periods 1..12 = recurring (pay Recurring Price),
  weighted by survival. The data supplies exactly 12 renewal transitions (C01..C1112).
- Payment-provider fee = 12% on ALL collected revenue (intro, recurring, upsells). Net = Gross * (1-0.12).
- Upsells are one-time at checkout; expected value = conv_rate * price, applied once to every subscriber.
- HORIZON: two readings are computed because the brief says "one-year horizon" but gives 12
  transitions per plan:
  (A) FULL-CURVE : apply all 12 transitions to every plan (matches the data given; for the
                    4-week plan 13 periods x 4wk = exactly 52 weeks). PRIMARY.
  (B) STRICT-52WK : count only payments landing within 52 calendar weeks
                    (12-week plan -> only 4 recurring periods; others unchanged). SENSITIVITY.
"""

FEE = 0.12

plans = {
    "1-WEEK": dict(mix=0.10, intro=6.93, rec=39.99, rec_wk=4, intro_wk=1,
                  C=[.55,.50,.60,.75,.80,.80,.80,.80,.80,.80,.80,.80],
                  up1=(0.30,1.99), up2=(0.12,0.99)),
    "4-WEEK": dict(mix=0.70, intro=19.99, rec=39.99, rec_wk=4, intro_wk=4,
                  C=[.67,.65,.70,.75,.80,.80,.80,.80,.80,.80,.80,.80],
                  up1=(0.30,69.99), up2=(0.12,29.99)),
    "12-WEEK": dict(mix=0.20, intro=39.99, rec=62.99, rec_wk=12, intro_wk=12,
                  C=[.64,.57,.65,.75,.75,.75,.75,.75,.75,.75,.75,.75],
                  up1=(0.30,69.99), up2=(0.12,29.99)),
}

def survivals(C):
    s, acc = [], 1.0
    for c in C:
        acc *= c
        s.append(acc)      # s[k-1] = survival to recurring period k
    return s

def periods_within_year(intro_wk, rec_wk):
    """Recurring payments CHARGED within the 52-week horizon (strict reading).
    Recurring period k is charged at week intro_wk + (k-1)*rec_wk; count it if that
    charge falls inside the year (LTV = cash collected, so charge date is what counts,
    not whether the service period extends past week 52). Capped at the 12 given transitions.
    -> 12-week plan: charges at wk 12/24/36/48 -> 4 periods; 4-week & 1-week -> 12."""
    n = 0
    for k in range(1, 13):
        if intro_wk + (k - 1) * rec_wk < 52:
            n += 1
        else:
            break
    return n

def recurring_revenue(p, mode):
    s = survivals(p["C"])
    if mode == "full":
        k = 12
    else:
        k = periods_within_year(p["intro_wk"], p["rec_wk"])
    return p["rec"] * sum(s[:k]), k
```

```

def upsell_ev(p):
    return p["up1"][0]*p["up1"][1] + p["up2"][0]*p["up2"][1]

def plan_ltv(p, mode):
    rec_rev, k = recurring_revenue(p, mode)
    sub_gross = p["intro"] + rec_rev
    up = upsell_ev(p)
    gross = sub_gross + up
    return dict(k=k, intro=p["intro"], rec_rev=rec_rev, upsell=up,
               gross=gross, net=gross*(1-FEE))

print("="*72)
print("TASK 1 – LTV per plan (1-year horizon). Fee = 12%.")
for mode, label in [("full", "(A) FULL-CURVE [PRIMARY]"), ("strict", "(B) STRICT-52WK [sensitivity]"]):
    print(f"\n--- Horizon reading {label} ---")
    print(f"{'plan':>8} {'mix':>5} {'recPds':>6} {'intro':>7} {'recRev':>8} {'upsell':>7} {'GROSS':>8} {'NET':>8}")
    blended = 0.0
    for name, p in plans.items():
        r = plan_ltv(p, mode)
        blended += p["mix"]*r["net"]
        print(f"{'name':>8} {p['mix']*100:4.0f}% {r['k']:6d} {r['intro']:7.2f} {r['rec_rev']:8.2f} {r['upsell']:7.2f} {r['gross']:8.2f} {r['net']:8.2f}")
    print(f"{'BLENDED NET LTV (mix-weighted)':>55}: ${blended:6.2f}")

print("\n"+"="*72)
print("TASK 2 – A/B: 4-week buyer offered Plan Upgrade ($49.99) INSTEAD of 2nd upsell.")
print("Upgrade buyer becomes a 12-week subscriber (12-week recurring economics).")
UP_PRICE = 49.99
for mode, label in [("full", "FULL-CURVE"), ("strict", "STRICT-52WK")]:
    p4, p12 = plans["4-WEEK"], plans["12-WEEK"]
    recRev4, _ = recurring_revenue(p4, mode)
    recRev12, _ = recurring_revenue(p12, mode)
    up2_4 = p4["up2"][0]*p4["up2"][1] # expected 2nd-upsell value being replaced
    # Break-even p*: p*(UP_PRICE + recRev12 - recRev4) = up2_4 (intro & 1st upsell cancel; fee uniform)
    denom = UP_PRICE + recRev12 - recRev4
    p_star = up2_4 / denom
    print(f"\n--- {label} ---")
    print(f" recurring rev 4-week = ${recRev4:6.2f}")
    print(f" recurring rev 12-week = ${recRev12:6.2f} (delta vs 4wk = ${recRev12-recRev4:+.2f})")
    print(f" 2nd-upsell value replaced (0.12*29.99) = ${up2_4:.2f}")
    print(f" each upgrade adds ${denom:.2f} gross; cannibalises ${up2_4:.2f}")
    print(f" >> BREAK-EVEN upgrade conversion p* = {p_star*100:.2f}%")
    # sample scenarios
    ctrl_gross = p4["intro"] + recRev4 + p4["up1"][0]*p4["up1"][1] + up2_4
    print(f" control 4-week gross LTV = ${ctrl_gross:.2f} (net ${ctrl_gross*(1-FEE):.2f})")
    for p in (0.05, 0.10, 0.20, 0.30):
        test_gross = (p4["intro"] + p4["up1"][0]*p4["up1"][1]
                     + (1-p)*recRev4 + p*(UP_PRICE + recRev12))
        lift = (test_gross-ctrl_gross)/ctrl_gross*100
        print(f" p={p*100:4.0f}% test gross LTV ${test_gross:7.2f} net ${test_gross*(1-FEE):7.2f} lift {lift:+5.1f}%")

```

Code · Part A — the segment clustering

```
#!/usr/bin/env python3
"""Part A – derive audience segments from the quiz data instead of asserting them.

WHY THIS EXISTS
`segment_sizer.py` applied a hand-written if/elif rule chain: the six segments were authored
from a jobs-to-be-done framework and then counted, with unmatched users falling through to a
default. That is a taxonomy, not a segmentation – the structure came from the analyst, not the
data, so it cannot be falsified. This script asks the opposite question: if nobody told the
data what the segments were, how many groups are actually there, and what are they?

METHOD
k-modes (Huang, 1998) on the categorical quiz answers.
  · Distance is Hamming – the count of questions two users answered differently.
  · Centroids are per-attribute MODES, not means. k-means is wrong here: a mean over nominal
    levels ("35-44", "Business owner") has no meaning, and one-hot + Euclidean silently
    asserts that every pair of levels is equidistant, which is the assumption under test.
  · Missing answers are kept as their own level. In a 26-step quiz funnel, *not answering*
    is behaviour (where someone dropped), not absent data – discarding it would bias the
    sample toward finishers.
  · k chosen by the elbow in within-cluster cost, cross-checked against interpretability.
  · Multiple restarts per k; the best-cost run wins. Seeded, so it reproduces exactly.

READ THE OUTPUT AS
cluster profiles are reported by LIFT (share inside the cluster ÷ share in the population),
not by raw share. The modal answer of a cluster is usually just the modal answer of the whole
population; what makes a cluster a cluster is what it over-represents.
"""
import collections
import csv
import os
import random
import sys

random.seed(42)
csv.field_size_limit(10 ** 7)

HERE = os.path.dirname(os.path.abspath(__file__))
SRC = os.path.join(HERE, "..", "materials", "quiz-funnel-answers", "Quiz Funnel Answers.csv")

# The questions that carry motive or identity. Chosen for coverage (>25k answers each) and
# because each maps to something a PM could act on. IDs are the `ID` column, not `question_id`
# (which is 0 for every row in this export).
CORE = {
    "391": "age",
    "357": "gender",
    "394": "work_status",
    "331": "learn_claude_for",
    "390": "benefit",
    "332": "ai_experience",
    "336": "career_fear",
    "335": "feels_ready",
    "1241": "used_claude",
    "1426": "field",
}

MIN_ANSWERED = 6          # users answering fewer than this are quiz drop-offs, not segments
NA = "(no answer)"

def load():
    """Pivot the long export (one row per user-question) into one row per user."""
    wide = collections.defaultdict(dict)
    with open(SRC, encoding="utf-8-sig") as f:
        for r in csv.DictReader(f):
            qid = r["ID"]
```

```

        if qid in CORE:
            a = r["question_answer"].strip()
            if a:
                wide[r["epv_user_id"]][CORE[qid]] = a
    cols = list(CORE.values())
    rows = [[u.get(c, NA) for c in cols] for u in wide.values()]
    kept = [r for r in rows if sum(v != NA for v in r) >= MIN_ANSWERED]
    return cols, rows, kept

def cost(rows, modes, assign):
    return sum(sum(a != b for a, b in zip(rows[i], modes[assign[i]])) for i in range(len(rows)))

def kmodes(rows, k, iters=12, restarts=4):
    best = None
    m = len(rows[0])
    for _ in range(restarts):
        modes = [list(r) for r in random.sample(rows, k)]
        assign = [0] * len(rows)
        for _ in range(iters):
            moved = False
            for i, r in enumerate(rows):
                d = [sum(a != b for a, b in zip(r, mo)) for mo in modes]
                new = d.index(min(d))
                if new != assign[i]:
                    assign[i] = new
                    moved = True
            for c in range(k):
                members = [rows[i] for i in range(len(rows)) if assign[i] == c]
                if not members:
                    modes[c] = list(random.choice(rows))
                    continue
                modes[c] = [collections.Counter(mem[j] for mem in members).most_common(1)[0][0]
                           for j in range(m)]
            if not moved:
                break
        c = cost(rows, modes, assign)
        if best is None or c < best[0]:
            best = (c, modes, assign[:])
    return best

def main():
    cols, allrows, rows = load()
    n = len(rows)
    print(f"quiz respondents in export          {len(allrows):,}")
    print(f"answered >= {MIN_ANSWERED} of {len(cols)} core questions  {n:,}  "
          f"({n/len(allrows)*100:.1f}%)  <- the clustering sample\n")

    base = [collections.Counter(r[j] for r in rows) for j in range(len(cols))]

    print("== choosing k: within-cluster Hamming cost ==")
    print("  k      cost    cost/user    improvement")
    prev = None
    results = {}
    for k in range(2, 9):
        c, modes, assign = kmodes(rows, k)
        results[k] = (c, modes, assign)
        imp = f"({prev-c)/prev*100:5.1f}%" if prev else "  -"
        print(f"  {k:>2}  {c:>7,}  {c/n:>8.3f}  {imp}")
        prev = c

    k = int(sys.argv[1]) if len(sys.argv) > 1 else 6
    c, modes, assign = results[k]
    sizes = collections.Counter(assign)
    print(f"\n== k = {k} · cluster profiles (lift = share in cluster / share in population) ==")
    for cl, sz in sizes.most_common():
        members = [rows[i] for i in range(n) if assign[i] == cl]

```

```
print(f"\n — cluster {cl}  n = {sz:}, ({sz/n*100:.1f}%")
marks = []
for j, col in enumerate(cols):
    cnt = collections.Counter(m[j] for m in members)
    for val, v in cnt.most_common(3):
        if val == NA:
            continue
        share = v / sz
        pop = base[j][val] / n
        if pop > 0 and share > .18:
            marks.append((share / pop, col, val, share))
for lift, col, val, share in sorted(marks, reverse=True)[:6]:
    print(f"      {lift:4.2f}x  {share*100:4.1f}%  {col:16} {val[:52]}")

if __name__ == "__main__":
    main()
```